

Learned Page Replacement under Workload Shift

Course: CSE-307 Operating Systems - Term Paper (Section B), Track 1

Name: Md Muhibuzzaman Khan | ID: 202414085

1. Problem framing

FIFO and LRU assume the access pattern stays roughly the same. Real programs change phase: a loop over a small working set can be followed by a scan or by almost random accesses. I wanted to see how much three classical policies (FIFO, LRU, Belady's Optimal) and a small learned policy suffer when the pattern changes in the middle of a run. The questions were: (a) can a lightweight model beat LRU on a stable pattern, (b) what happens to it after the shift, and (c) does retraining it online help.

2. Implementation summary

All code is in Python (*src/*). The simulator walks through the trace with a fixed number of frames (32) and counts a hit or a page fault for each access. FIFO evicts the oldest loaded page, LRU the page with the oldest last access, and Optimal the page whose next use is farthest in the future (checked against the textbook string 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 with 3 frames: 15 / 12 / 9 faults).

Learned policy. Whenever a fault happens with a full memory, each resident page gets four features: time since last access, number of accesses in the last 200 references, total accesses so far, and time since it was loaded. A classifier predicts whether the page is *not* reused in the next 64 accesses (label 1 = good to evict, labels come from the future of the trace in hindsight, in the spirit of Belady). The page with the highest probability is evicted. I tried a depth-4 decision tree and logistic regression trained offline on a pre-shift trace (*static*), a decision tree trained on a trace that already contains the shift (*mixed-trained*, used as an ablation), and a decision tree that starts as LRU and is retrained every 100 evictions on recent decisions whose outcome is already known (*online*).

3. Experimental setup

Each trace has 10,000 accesses over 300 pages. In the first 5,000 accesses about 70% of references go to a small hot set of 20 pages (Zipf-like, slowly drifting) and about 30% are sequential scans (20-60 pages long) through a cold region. In the last 5,000 accesses 90% of events are single uniform random accesses over all 300 pages and 10% are short bursts (4-9 accesses on 3-6 pages). Models were trained on separate traces (different seeds). Everything was repeated for 10 seeds; I report the mean and the standard deviation. Hit ratio and fault counts are computed separately for the two halves (5,000 accesses each). The hit ratio before the shift includes the cold-start misses.

4. Results

Policy	Hit ratio before	Hit ratio after	Drop (pp)	Faults before	Faults after
FIFO	0.619 ± 0.047	0.282 ± 0.009	33.7	1907 ± 237	3592 ± 46
LRU	0.622 ± 0.047	0.282 ± 0.009	34.0	1891 ± 236	3592 ± 46
Optimal	0.736 ± 0.034	0.510 ± 0.008	22.6	1321 ± 169	2451 ± 38
Learned-DT (static)	0.719 ± 0.036	0.255 ± 0.018	46.4	1407 ± 178	3726 ± 91
Learned-LR (static)	0.724 ± 0.035	0.198 ± 0.010	52.6	1382 ± 176	4010 ± 48
Learned-DT (online)	0.701 ± 0.035	0.263 ± 0.010	43.8	1495 ± 174	3685 ± 51
Learned-DT (mixed-trained)	0.719 ± 0.036	0.220 ± 0.031	49.9	1405 ± 180	3902 ± 154

Table 1: hit ratio and page faults per 5,000 accesses (mean ± std over 10 seeds, 32 frames).

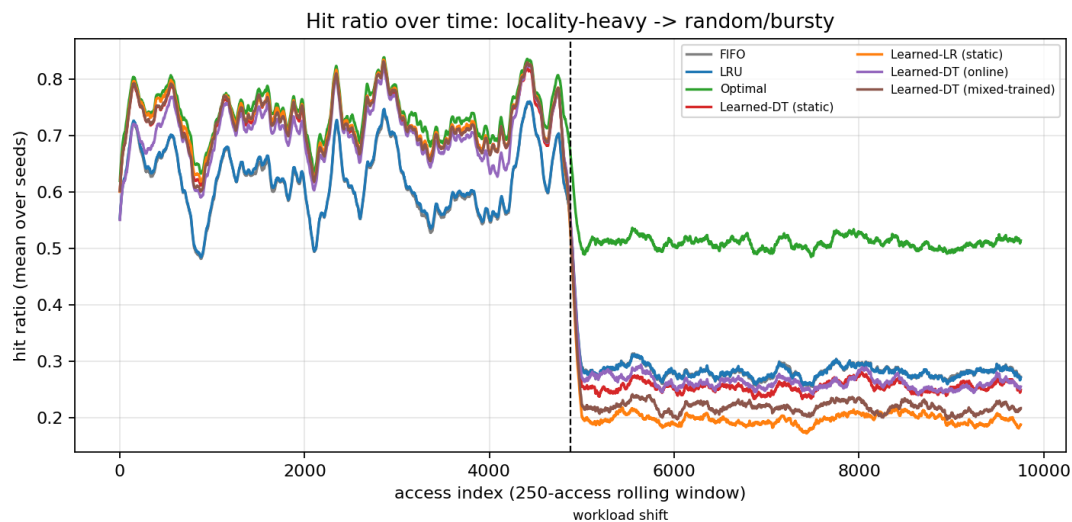


Figure 1: rolling hit ratio (250 accesses), averaged over seeds. FIFO and LRU lie on top of each other.

5. Analysis: what changed under the shift

FIFO and LRU became the same. After the workload shift, the performance of FIFO and LRU became equal (hit ratio 0.282, about 3,592 faults for both). The accesses became random, so the recently used pages no longer tell which page will be needed next, and LRU has no advantage over FIFO.

The learned models got worse. The models were trained on the old workload, but the access pattern changed. Therefore, their predictions became less accurate than LRU. After the shift, static DT dropped to 0.255 and static LR to 0.198, both below LRU (0.282), so the learned policies degraded the most (drops of 46.4 and 52.6 points against 34.0 for LRU).

Online retraining helped only a little. Retraining allowed the model to learn the new workload and the hit ratio improved to 0.263 (static DT: 0.255). However, in the random pattern pages are rarely reused soon, so there is little pattern to learn and the improvement was limited. It stayed below LRU (0.282).

6. Bonus: explanation confidence

For each eviction a template generates a sentence such as *'evicting page 8; last used 71 accesses ago, touched once in the last 200 accesses; expect no reuse within 64 accesses; confidence 0.98'* (samples in *results/explanations_sample.txt*). The confidence is the model's probability for the victim, and a decision counts as correct if the page is really not reused within 64 accesses. For the static DT, confidence is high before the shift (0.99 when correct, accuracy 98%) but after the shift it is about 0.58 and is not higher on correct decisions (0.579 correct vs 0.592 wrong), so it is poorly calibrated under shift (ECE 0.23). The online tree is better calibrated (ECE 0.09, 0.93 vs 0.92 mean confidence correct/wrong), though the gap between correct and wrong is small. Note that 'correct' is an easy condition in the random half, so high accuracy there does not mean a high hit ratio.

7. Limitations

The traces are synthetic and the results depend on the chosen mix, the number of frames and the 64-access horizon. The features are simple on purpose; richer features (for example scan detection) or a change detector would probably help. Code, charts and tables: see the GitHub repository (README).